Hello World Spring Boot on AWS

Class Project Presentation

Author: Sanjay Akula
Date: July 27, 2026
Project: `helloworld-springboot-java`

Table of Contents

---

1. Project Overview

This project demonstrates how to build, containerize, and deploy a **Java Spring Boot** web application to **Amazon Web Services (AWS)** using modern DevOps practices.

What it does

- Exposes a simple REST API that returns a "Hello World" JSON response
- Shows the hostname of the server it's running on (useful for verifying load balancing)
- Reports its deployment environment (local, ECS-Fargate, or EC2)
- Provides a health check endpoint used by the AWS load balancer

Technology Stack

Layer	Technology
Language	Java 17
Framework	Spring Boot 3.3.2
Build tool	Apache Maven
Container	Docker (multi-stage build)
Cloud	Amazon Web Services (AWS)
Infrastructure	Terraform >= 1.5
CI/CD	AWS CodePipeline + CodeBuild

Project Structure

```
helloworld-springboot-java/
├── src/                    ← Java application source code
├── Dockerfile              ← Container build instructions
├── pom.xml                 ← Maven build config (Java 17, Spring Boot 3.3.2)
├── buildspec.yml           ← CodeBuild CI/CD instructions
├── terraform/              ← ECS Fargate infrastructure (IaC)
├── terraform-ec2/          ← EC2 infrastructure (IaC)
├── codepipeline/           ← CI/CD pipeline infrastructure (IaC)
├── ec2/                    ← EC2 deploy scripts
└── ecs/                    ← ECS task definition reference
```

---

2. Application Design

The application has two source files and one config file.

Entry Point — `HelloWorldApplication.java`

```java
package com.example.helloworld;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class HelloWorldApplication {
    public static void main(String[] args) {
        SpringApplication.run(HelloWorldApplication.class, args);
    }
}
```

This is the standard Spring Boot entry point. The `@SpringBootApplication` annotation enables component scanning, auto-configuration, and property support.

REST Controller — `HelloController.java`

```java
@RestController
public class HelloController {

    @GetMapping("/")
    public Map<String, String> hello() {
        Map<String, String> response = new LinkedHashMap<>();
        response.put("message", "Hello World from AWS!");
        response.put("host", getHostName());
        response.put("deployment", System.getenv().getOrDefault("DEPLOYMENT_TYPE",
"local"));
        response.put("version", "1.0.0");
        return response;
```

```
    }

    @GetMapping("/hello")
    public String helloText() {
        return "Hello World! Running on: " + getHostName();
    }

    @GetMapping("/health")
    public Map<String, String> health() {
        Map<String, String> response = new LinkedHashMap<>();
        response.put("status", "UP");
        response.put("host", getHostName());
        return response;
    }
}
```

API Endpoints

Method	Path	Response	Purpose
GET	`/`	JSON	Main hello world response
GET	`/hello`	Plain text	Simple text response
GET	`/health`	JSON `{"status":"UP"}`	ALB health check target
GET	`/actuator/health`	JSON	Spring Boot built-in health

Sample Response from `/`

```
{
  "message": "Hello World from AWS!",
  "host": "ip-10-0-1-45.ec2.internal",
  "deployment": "ECS-Fargate",
  "version": "1.0.0"
}
```

The `host` field changes per container/instance, proving the load balancer is routing to different backends.

`application.properties`

```
server.port=8080
spring.application.name=helloworld-springboot-java

# Expose health and info endpoints for ALB health checks
management.endpoints.web.exposure.include=health,info
management.endpoint.health.show-details=always
```

Maven Dependencies (`pom.xml`)

Dependency	Purpose
`spring-boot-starter-web`	Embedded Tomcat + REST support
`spring-boot-starter-actuator`	Health/info endpoints
`spring-boot-starter-test`	JUnit testing

---

3. Containerization with Docker

The application is packaged into a Docker image using a **multi-stage build** — an industry best practice that keeps the final image small and secure.

`Dockerfile`

```
# ---- Stage 1: Build ----
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn -q dependency:go-offline        # cache dependencies first
COPY src ./src
RUN mvn -q clean package -DskipTests   # compile + package the JAR

# ---- Stage 2: Run ----
FROM eclipse-temurin:17-jre             # much smaller than the build image
WORKDIR /app
RUN groupadd -r appgroup && useradd -r -g appgroup appuser  # non-root user
COPY --from=build /app/target/helloworld-springboot-java.jar app.jar
RUN chown appuser:appgroup app.jar
USER appuser                            # run as non-root for security
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Why Multi-Stage?

Without multi-stage	With multi-stage
Image includes Maven, JDK, source code	Image only has JRE + compiled JAR
~700 MB image size	~250 MB image size
Larger attack surface	Smaller attack surface

Security: Non-Root User

Running the app as a non-root user inside the container is a security best practice. Even if the container is compromised, the attacker doesn't have root privileges on the host.

---

4. AWS Infrastructure — Option A: ECS Fargate

ECS Fargate is a serverless container service — AWS manages the underlying servers. You only define the container and how much CPU/RAM it needs.

Architecture

```
Internet
    │
    ▼
[Application Load Balancer]   ← distributes traffic, does health checks
    │
    ▼
[ECS Fargate Service]
  ├── Task (Container) 1  ← runs on AWS-managed infrastructure
  ├── Task (Container) 2
  └── Task (Container) N
    │
    ▼
[Amazon ECR]              ← stores Docker images
[Amazon S3]               ← stores built JARs
[CloudWatch Logs]         ← container logs
```

AWS Services Used

Service	Role
VPC	Isolated network with public subnets
ALB	Application Load Balancer — routes HTTP traffic
ECS Cluster	Logical group of container tasks
ECS Task Definition	Blueprint: image, CPU, memory, env vars
ECS Service	Keeps N tasks running, registers with ALB
ECR	Elastic Container Registry — private Docker registry
S3	Stores built JAR artifacts
CloudWatch	Container logs with 7-day retention
IAM	Task execution role (ECR pull + CloudWatch write)

ECS Task Configuration

```
cpu    = 256   # 0.25 vCPU
memory = 512   # 512 MB RAM
```

This is enough for a demo app — costs fractions of a cent per hour.

Health Check Flow

```
ALB checks GET /health every 30 seconds
    │
    ├── Returns HTTP 200 → task is healthy, receives traffic
    └── Returns non-200 → task is replaced automatically
```

---

5. AWS Infrastructure — Option B: EC2

For this option, the application runs on a traditional virtual machine (EC2 instance) instead of a serverless container service.

Option B1: Docker on EC2 (via Terraform)

Terraform provisions:

- A **t3.micro** EC2 instance (free tier eligible)
- An **Application Load Balancer** in front of it
- An **IAM instance profile** so the EC2 can pull from ECR and access S3
- A **user-data script** that runs on first boot to pull and start the Docker container

User-Data Bootstrap Script (runs on first EC2 boot)

```
#!/bin/bash
yum update -y
yum install -y docker
systemctl enable --now docker
# Log in to ECR and pull the image
aws ecr get-login-password --region ${region} \
  | docker login --username AWS --password-stdin ${ecr_url}
docker pull ${ecr_image}
# Run the container
docker run -d \
  --name helloworld \
  -p 8080:8080 \
  -e DEPLOYMENT_TYPE=EC2-Docker \
  --restart unless-stopped \
  ${ecr_image}
```

Option B2: JAR on EC2 (no Docker)

Runs the Spring Boot JAR directly on the EC2 instance as a **systemd service** — no Docker involved.

```
[EC2 Instance]
└── systemd
    └── helloworld-springboot.service
        └── java -jar helloworld-springboot-java.jar
```

ECS Fargate vs EC2 Comparison

Feature	ECS Fargate	EC2
Server management	None (serverless)	You manage the OS

Scaling	Automatic	Manual or Auto Scaling Group
Cost model	Per task second	Per instance hour
Cold start	~30–60 seconds	Instance always running
Best for	Variable/unknown load	Predictable steady load

---

6. CI/CD Pipeline (CodePipeline + CodeBuild)

The CI/CD pipeline automates the entire path from code commit to production deployment.

Pipeline Flow

```
Developer pushes to GitHub (main branch)
          │
          ▼
   [GitHub Webhook]  ← fires immediately on push (no polling delay)
          │
          ▼
   [CodePipeline]
          │
   ┌──────┴──────┐
   │  Stage 1:   │
   │   Source    │  GitHub repo → S3 artifact store
   └──────┬──────┘
          │
   ┌──────┴──────┐
   │  Stage 2:   │
   │    Build    │  CodeBuild runs buildspec.yml:
   │             │  1. mvn test       ← run unit tests
   │             │  2. mvn package    ← build JAR
   │             │  3. docker build   ← build image
   │             │  4. docker push    ← push to ECR
   │             │  5. aws s3 cp      ← upload JAR to S3
   │             │  6. write imagedefinitions.json
   └──────┬──────┘
          │
   ┌──────┴──────┐
   │  Stage 3:   │
   │   Deploy    │  ECS rolling deploy using imagedefinitions.json
   └─────────────┘
```

`buildspec.yml` — The Build Recipe

```yaml
phases:
  pre_build:
    commands:
      - aws ecr get-login-password | docker login ...   # authenticate to ECR
      - IMAGE_TAG=$(echo $CODEBUILD_RESOLVED_SOURCE_VERSION | cut -c 1-7)
```

```
  build:
    commands:
      - mvn -q test                          # run tests first
      - mvn -q clean package -DskipTests     # build the JAR
      - docker build -t $REPOSITORY_URI:latest -t $REPOSITORY_URI:$IMAGE_TAG .

  post_build:
    commands:
      - docker push $REPOSITORY_URI:latest
      - docker push $REPOSITORY_URI:$IMAGE_TAG
      - aws s3 cp target/*.jar s3://$S3_ARTIFACT_BUCKET/releases/$IMAGE_TAG/
      - printf '[{"name":"%s","imageUri":"%s"}]' $CONTAINER_NAME
$REPOSITORY_URI:$IMAGE_TAG \
          > imagedefinitions.json             # tells ECS which image to deploy
```

`imagedefinitions.json`

This small file is the handshake between CodeBuild and the ECS Deploy stage:

```
[
  {
    "name": "helloworld-springboot",
    "imageUri": "123456789.dkr.ecr.us-east-1.amazonaws.com/helloworld-
springboot:a1b2c3d"
  }
]
```

ECS reads this file and performs a rolling deployment — replacing old containers with the new image one at a time, with zero downtime.

Image Tagging Strategy

Every build creates two image tags:

- `:latest` — always points to the most recent build
- `:<git-sha>` — e.g., `:a1b2c3d` — immutable tag for rollbacks

---

7. Infrastructure as Code (Terraform)

All AWS infrastructure is defined in **Terraform** — meaning the infrastructure is reproducible, version-controlled, and can be created or destroyed with a single command.

Terraform Modules

```
terraform/           ← ECS Fargate infrastructure
├── main.tf          ← Provider config + S3 remote state backend
├── variables.tf     ← Input parameters (region, image, CPU, etc.)
├── network.tf       ← VPC, subnets, internet gateway, route tables
├── security_groups.tf ← Firewall rules for ALB and ECS tasks
├── ecr.tf           ← ECR container registry
```

```
├── s3.tf            ← S3 artifact bucket
├── alb.tf           ← Application Load Balancer + target group
├── iam.tf           ← ECS execution role + policies
├── ecs.tf           ← ECS cluster + task definition + service
└── outputs.tf       ← Printed values after apply (ALB URL, ECR URL)
```

Remote State Backend

Terraform state is stored in **S3** (not locally), so multiple team members can collaborate safely:

```
backend "s3" {
  bucket         = "helloworld-springboot-tfstate"
  key            = "ecs/terraform.tfstate"
  region         = "us-east-1"
  encrypt        = true
  dynamodb_table = "helloworld-springboot-tfstate-lock"  ← prevents simultaneous
applies
}
```

The **DynamoDB table** acts as a lock — if two people run `terraform apply` at the same time, only one proceeds.

Key Terraform Resources

```
# VPC and networking
resource "aws_vpc" "main" { cidr_block = "10.0.0.0/16" }

# Load balancer
resource "aws_lb" "main" {
  name               = "helloworld-alb"
  load_balancer_type = "application"
  subnets            = aws_subnet.public[*].id
}

# ECS Fargate service
resource "aws_ecs_service" "app" {
  name          = "helloworld-springboot-service"
  cluster       = aws_ecs_cluster.main.id
  desired_count = 2          ← keep 2 containers running
  launch_type   = "FARGATE"
}
```

Deploy with 3 Commands

```
cd terraform
terraform init     # download AWS provider
terraform apply    # create all resources
terraform destroy  # tear everything down (clean up, avoid charges)
```

---

8. GitHub Actions Workflows

In addition to CodePipeline, the project includes **GitHub Actions** for automated checks on every pull request.

Workflows

File	Trigger	What it does
`docker-build-push.yml`	Push to main	Builds Docker image, pushes to ECR
`deploy.yml`	Push to main	Full build + ECS deploy

These provide an alternative CI/CD path using GitHub's built-in automation instead of AWS CodePipeline.

---

9. Security Highlights

Security was considered at every layer of this project:

Layer	Practice
Docker	Non-root user inside container
Docker	Multi-stage build (no build tools in production image)
S3	Versioning + AES-256 encryption at rest
S3	Public access blocked
ECR	Private registry, image scanning enabled
IAM	Least-privilege roles (ECS task role only has what it needs)
Terraform state	Encrypted S3 bucket + DynamoDB lock
GitHub token	Marked `sensitive = true` in Terraform, never logged
Networking	ALB security group only allows HTTP (80); ECS tasks only allow traffic from ALB

Security Group Rules

```
ALB Security Group:
  Inbound:  0.0.0.0/0  → port 80  (public HTTP)
  Outbound: all

ECS Tasks Security Group:
  Inbound:  ALB SG only → port 8080  (only ALB can reach tasks)
  Outbound: all (for ECR pulls, CloudWatch logs)
```

---

10. How to Run the Project

Prerequisites

- AWS CLI configured (`aws configure`)
- Terraform >= 1.5
- Java 17 + Maven
- Docker

Step 1 — Run Locally

```
mvn clean package
java -jar target/helloworld-springboot-java.jar

curl http://localhost:8080/
# {"message":"Hello World from AWS!","host":"your-
mac","deployment":"local","version":"1.0.0"}
```

Step 2 — Bootstrap Terraform Backend (once)

```
aws s3api create-bucket --bucket helloworld-springboot-tfstate --region us-east-1
aws dynamodb create-table \
  --table-name helloworld-springboot-tfstate-lock \
  --attribute-definitions AttributeName=LockID,AttributeType=S \
  --key-schema AttributeName=LockID,KeyType=HASH \
  --billing-mode PAY_PER_REQUEST --region us-east-1
```

Step 3 — Deploy to ECS Fargate

```
cd terraform
terraform init && terraform apply

# Push Docker image to ECR
REPO_URL=$(terraform output -raw ecr_repository_url)
docker build -t $REPO_URL:latest .
docker push $REPO_URL:latest

# Trigger ECS deployment
aws ecs update-service \
  --cluster helloworld-springboot-cluster \
  --service helloworld-springboot-service \
  --force-new-deployment --region us-east-1

# Test
curl $(terraform output -raw alb_dns_name)
```

Step 4 — Set Up CI/CD

```
cd codepipeline
terraform apply -var="github_oauth_token=<your-token>"
# From now on, every git push to main triggers a full build + deploy
```

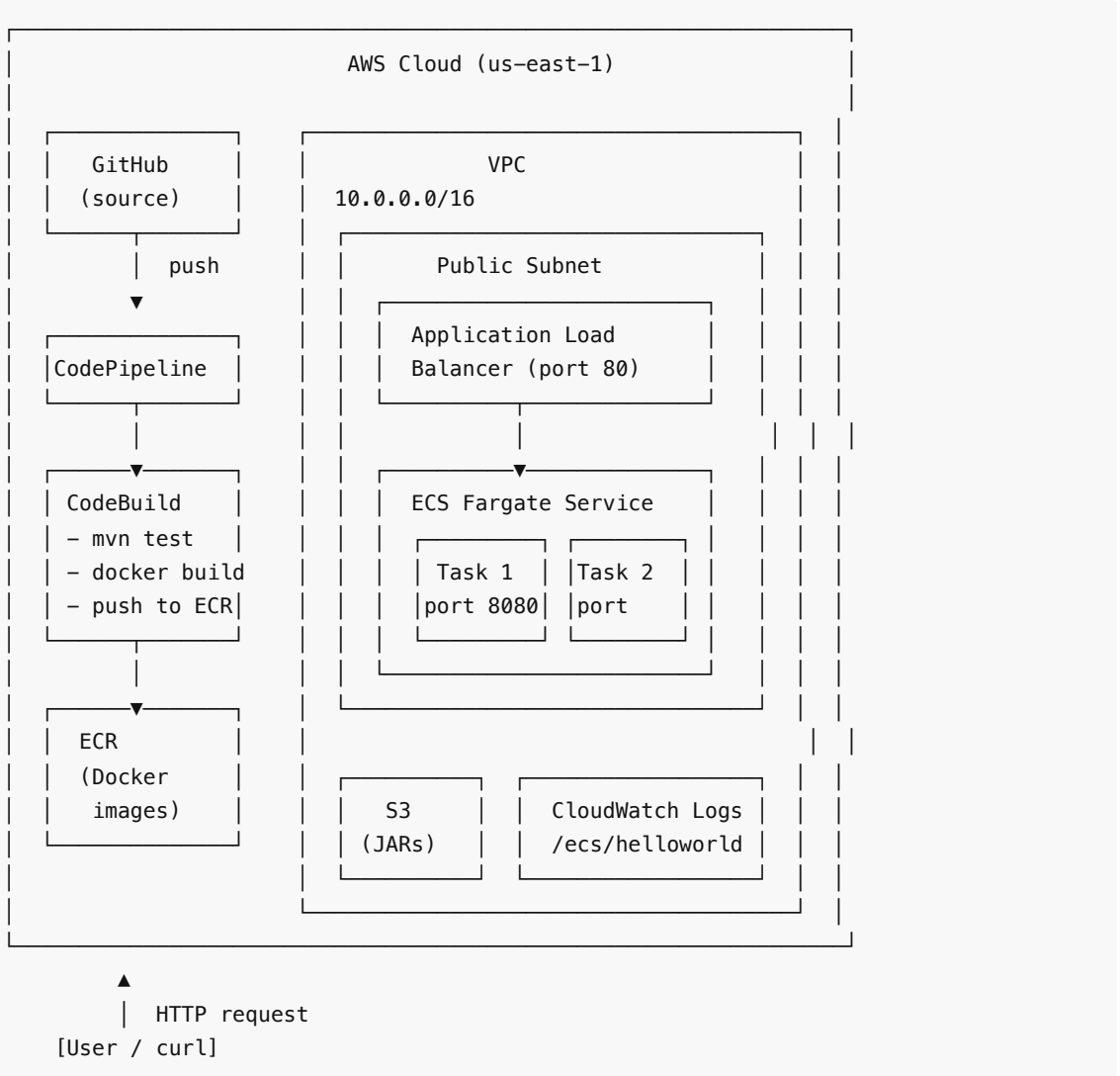
Step 5 — Tear Down (avoid charges)

```
terraform destroy    # from terraform/, terraform-ec2/, and codepipeline/
```

---

11. Architecture Diagram

Full System Architecture



---

12. Key Learnings & Summary

What This Project Demonstrates

Spring Boot: Building a production-ready REST API with health checks, actuator endpoints, and environment-aware configuration.

Docker: Multi-stage builds to produce small, secure container images. Running as a non-root user.

AWS ECS Fargate: Serverless container deployment with automatic scaling, load balancing, and zero-downtime rolling deployments.

AWS EC2: Traditional VM deployment with Docker, systemd service management, and IAM instance profiles.

Terraform: Infrastructure as Code — reproducible, version-controlled cloud infrastructure. Remote state management with S3 and DynamoDB locking.

CI/CD: Automated pipeline from GitHub push to production deployment using CodePipeline, CodeBuild, and webhooks. Every commit is tested, built, containerized, and deployed automatically.

Security: Defense in depth — non-root containers, encrypted state, private registries, least-privilege IAM, network-level isolation.

Two Deployment Options Compared

```
Option A (ECS Fargate):          Option B (EC2):
- Serverless                     - Full control over OS
- AWS manages infra              - You manage the instance
- Scales automatically           - Manual scaling
- Great for microservices        - Great for steady workloads
- Higher abstraction             - More familiar to beginners
```

The CI/CD Value Proposition

Without CI/CD:

1. Developer writes code
2. Manually builds JAR
3. Manually builds Docker image
4. Manually pushes to ECR
5. Manually updates ECS service
6. ~15 minutes, error-prone

With CI/CD (this project):

1. Developer runs `git push`
2. Everything else happens automatically in ~5 minutes

Document generated from project source code — `helloworld-springboot-java`
Spring Boot 3.3.2 | Java 17 | AWS ECS Fargate + EC2 | Terraform | CodePipeline